

1. Logika cyfrowa

| Układ              | Formuła / sygnatura                                                 | Opis                                                                                                                  |
|--------------------|---------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Sumatory           |                                                                     |                                                                                                                       |
| Półsumator         | $s = x \oplus y$<br>$c = x \& y$                                    | Dodaje dwa bity. <b>s</b> to suma, <b>c</b> to bit przeniesienia (carry).                                             |
| Pełny sumator (FA) | $s = x \oplus y \oplus ci$<br>$co = x \& y \mid ci \& (x \oplus y)$ | Jak półsumator, ale przyjmuje też przeniesienie wejściowe <b>ci</b> .                                                 |
| Sumator n-bitowy   | $n \times \text{FA}$                                                | Kaskada (ripple-carry): wyjście $C_i$ trafia na wejście kolejnego FA. Ostatnie <b>co</b> to <b>Carry Out</b> całości. |

|                     |                         |                                                                                                                          |
|---------------------|-------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Układy kombinacyjne |                         |                                                                                                                          |
| Dekoder             | $n \rightarrow 2^n$     | Wejście koduje liczbę $k$ . Na wyjściu zapalony jest <b>tylko</b> bit nr $k$ .                                           |
| Multiplekser        | $2^n + n \rightarrow 1$ | $2^n$ bitów danych + $n$ bitów sterujących $S$ . Na wyjście przechodzi $S$ -ty bit danych.                               |
| ALU                 | $A, B, f \rightarrow C$ | Dekoder na bitach $f$ wybiera operację (np. $A + B$ , $A \mid B$ , $A \& B$ ); multipleksowanie wyników bramkami AND/OR. |

|                             |                                  |                                                                                                                                                    |
|-----------------------------|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Układy sekwencyjne (pamięć) |                                  |                                                                                                                                                    |
| Przerzutnik S-R             | <b>S</b> - set, <b>R</b> - reset | Przechowuje 1 bit ( $Q$ ). <b>S=1</b> ustawia $Q = 1$ , <b>R=1</b> zeruje, <b>S=R=0</b> trzyma stan. Zbudowany z dwóch sprzężonych <b>NOR</b> -ów. |
| Sterowany poziomem          | aktywny gdy <b>CLK</b> = 1       | Wejścia <b>S</b> / <b>R</b> bramkowane AND-em z zegarem. Stan zmienia się tylko przy <b>CLK=1</b> .                                                |
| Sterowany zboczem           | zbocze 1 $\rightarrow$ 0         | Dwa przerzutniki poziomowe w kaskadzie (master-slave), drugi z zanegowanym zegarem. Stan przepisuje się w <b>momencie</b> zmiany zegara.           |

2. Typy danych

| Typ    | char | short | int | long | float | double | void* |
|--------|------|-------|-----|------|-------|--------|-------|
| x86-64 | 1    | 2     | 4   | 8    | 4     | 8      | 8     |

3. Kodowanie liczb i konwersja

$$\text{B2U}(X) = \sum_{i=0}^{w-1} x_i 2^i \quad \text{B2T}(X) = -x_{w-1} 2^{w-1} + \sum_{i=0}^{w-2} x_i 2^i$$

$$\text{T2U}(x) = x < 0 ? x + 2^w : x \quad \text{U2T}(u) = u > \text{TMax} ? u - 2^w : u$$

Skróty: **B** = bity, **U** = unsigned, **T** = ze znakiem (kod U2).  
Stąd **B2U** = bity  $\rightarrow$  unsigned, podobnie **U2T**, **UMax**, **TMax**, **UAdd**, **TAdd**.

| Stała | Bity          | Wartość                          |
|-------|---------------|----------------------------------|
| UMax  | <b>11...1</b> | $2^w - 1$                        |
| TMax  | <b>01...1</b> | $2^{w-1} - 1$ ( <b>INT_MAX</b> ) |
| TMin  | <b>10...0</b> | $-2^{w-1}$ ( <b>INT_MIN</b> )    |
| −1    | <b>11...1</b> | te same bity co UMax             |

Promocja w C: **unsigned** i **int** w jednym wyrażeniu/porównaniu ( $< > ==$ )  $\rightarrow$  **int** promowany do **unsigned** (bity bez zmian,  $-1 \rightarrow \text{UMax}$ ).

4. Rozszerzanie, obcinanie i przesunięcia

|                                |                                               |
|--------------------------------|-----------------------------------------------|
| Rozszerz. 0 ( <b>zext</b> )    | <b>unsigned</b> : dopisuje 0 z góry           |
| Rozszerz. znak ( <b>sext</b> ) | <b>signed</b> : powiela bit znaku (MSB)       |
| <b>x &lt;&lt; k</b>            | $x \cdot 2^k$ (sgn./uns.)                     |
| <b>u &gt;&gt; k</b>            | $\lfloor u/2^k \rfloor$ — logiczne (zera)     |
| <b>x &gt;&gt; k</b>            | arytmetyczne (kopia MSB), zaokr. do $-\infty$ |
| $x/2^k$ do 0                   | <b>(x + (1&lt;&lt;k)-1) &gt;&gt; k</b>        |

Strength red.: **x\*24**  $\rightarrow$  **(x<<5)-(x<<3)**.

5. Operacje, overflow i endianness

|                                 |                                                    |
|---------------------------------|----------------------------------------------------|
| UAdd/TAdd                       | $(u + v) \bmod 2^w$                                |
| Nadmiar (+)                     | $u, v > 0$ , a wynik $< 0$ (wynik $-2^w$ )         |
| Nadmiar (-)                     | $u, v < 0$ , a wynik $\geq 0$ (wynik $+2^w$ )      |
| Negacja U2                      | <b>-x = ~x+1</b> ; <b>-TMin=TMin</b> , <b>-0=0</b> |
| Wykr. nadmiaru ( <b>s=x+y</b> ) | <b>((s^x)&amp;(s^y))&gt;&gt;(w-1)</b>              |
| Maska / abs                     | <b>m=x&gt;&gt;31; abs=(x^m)-m</b>                  |

| Schemat       | Najniższy adres | <b>0x0123</b>   |
|---------------|-----------------|-----------------|
| Big Endian    | MSB             | <b>[01][23]</b> |
| Little Endian | LSB             | <b>[23][01]</b> |

6. Zmiennoprzecinkowe (IEEE 754)

$$V = (-1)^s \times M \times 2^E \qquad \text{Bias} = 2^{k-1} - 1$$

| Format | bity | s | exp (k) | frac (n) |
|--------|------|---|---------|----------|
| float  | 32   | 1 | 8       | 23       |
| double | 64   | 1 | 11      | 52       |

Kategorie wartości

| Kategoria      | exp                                  | Własności                                                                                                                                                          |
|----------------|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Znormalizowane | $\neq 0 \dots 0$<br>$\neq 1 \dots 1$ | $E = \text{exp} - \text{Bias}$ . $M = 1.\text{frac}$ .<br>Najgęściej ułożone wokół 0.                                                                              |
| Zdenormaliz.   | $= 0 \dots 0$                        | $E = 1 - \text{Bias}$ . $M = 0.\text{frac}$ .<br>Reprezentują +0.0 i -0.0 (frac = 0)<br>oraz liczby bardzo bliskie zera.                                           |
| Specjalne      | $= 1 \dots 1$                        | Gdy $\text{frac} = 0$ , to $+\infty$ lub $-\infty$<br>(nadmiar/dzielenie przez 0).<br>Gdy $\text{frac} \neq 0$ , to NaN (np. $\sqrt{-1}$ ,<br>$\infty - \infty$ ). |

Zaokrąglenie GRS i Round-to-Even

Domyślny tryb to **Round-to-Even** (zapobiega błędowi statycznemu przy sumowaniu).



**G** - ostatni zachowany, **R** - pierwszy usunięty, **S** - OR reszty

| Warunek    | Ułamek  | Akcja                                                             |
|------------|---------|-------------------------------------------------------------------|
| $R=0$      | $< 0.5$ | W dół (odcięcie)                                                  |
| $R=1, S=1$ | $> 0.5$ | W górę (+1 do <b>G</b> )                                          |
| $R=1, S=0$ | $= 0.5$ | <b>Do parzystej:</b> w górę gdy <b>G=1</b> , w dół gdy <b>G=0</b> |

Własności matematyczne i rzutowanie

- **Brak łączności:**  $(a + b) + c \neq a + (b + c)$ , przez zaokrąglenia i utratę danych.
- **Brak rozdzielności:**  $a(b + c) \neq ab + ac$ .
- **Rzutowanie `int` → `float`:** Może stracić precyzję (zaokrągła zgodnie z trybem).
- **Rzutowanie `int` → `double`:** Dokładne i bezstratne (bo 52 bity mantysy > 32 bity inta).
- **Rzutowanie `float/double` → `int`:** Obcina ułamek w stronę 0. Jeśli poza zakresem lub NaN → z reguły zwraca **TMin** (`0x80000000`).

7. Rejestry x86\_64

|                   |                                                          |                   |                                                          |
|-------------------|----------------------------------------------------------|-------------------|----------------------------------------------------------|
| <code>%rax</code> | <code>%eax</code><br><code>%ah</code> <code>%al</code>   | <code>%rbx</code> | <code>%ebx</code><br><code>%bh</code> <code>%bl</code>   |
| <code>%rcx</code> | <code>%ecx</code><br><code>%ch</code> <code>%cl</code>   | <code>%rdx</code> | <code>%edx</code><br><code>%dh</code> <code>%dl</code>   |
| <code>%rsi</code> | <code>%esi</code><br><code>%sil</code>                   | <code>%rdi</code> | <code>%edi</code><br><code>%dil</code>                   |
| <code>%rbp</code> | <code>%ebp</code><br><code>%bp</code> <code>%bpl</code>  | <code>%rsp</code> | <code>%esp</code><br><code>%spl</code>                   |
| <code>%r8</code>  | <code>%r8d</code><br><code>%r8w</code> <code>%r8b</code> | <code>%r9</code>  | <code>%r9d</code><br><code>%r9w</code> <code>%r9b</code> |

**Uwaga:** Rejestry od `%r10` do `%r15` używają schematu:  
`%r10`, `%r10d`, `%r10w`, `%r10b`.

Podział ról rejestrów

|                                                                                                                                 |                                                                                |
|---------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| <code>%rax</code>                                                                                                               | Akumulator. Główny rejestr arytmetyczny. Przechowuje <b>wartość zwracaną</b> . |
| <code>%rdi</code> ,<br><code>%rsi</code> ,<br><code>%rdx</code> ,<br><code>%rcx</code> ,<br><code>%r8</code> , <code>%r9</code> | Kolejno: <b>od 1. do 6. argumentu funkcji</b> . Caller-saved.                  |
| <code>%rsp</code>                                                                                                               | Wskaźnik stosu (SP). Wskazuje wierzchołek ramki.                               |
| <code>%rbp</code>                                                                                                               | Wskaźnik bazy (BP). Początek ramki stosu. Callee-saved.                        |
| <code>%rbx</code> ,<br><code>%r12-<code>%r15</code></code>                                                                      | Ogólnego przeznaczenia. Wszystkie <b>callee-saved</b> .                        |
| <code>%rip</code>                                                                                                               | Wskaźnik instrukcji (Program Counter).                                         |

Rejestry wektorowe (zmiennoprzecinkowe)

|                                           |                                                                                                                                                  |
|-------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>%xmm0</code> do <code>%xmm15</code> | 128-bitowe rejestry. <code>%xmm0</code> to wartość zwracana ( <code>float</code> / <code>double</code> ). Pierwsze 8 to argumenty. Caller-saved. |
| <code>%ymm0</code> do <code>%ymm15</code> | 256-bitowe rozszerzenie rejestrów XMM. Dzielą z nimi najmłodsze 128 bitów.                                                                       |

8. Tryby adresowania (operands)

| Tryb                   | Format                      | Obliczanie adresu                   |
|------------------------|-----------------------------|-------------------------------------|
| Natychniastowy (Imm)   | <code>\$Imm</code>          | <code>Imm</code>                    |
| Rejestrowy (Reg)       | <code>%Ra</code>            | <code>%Ra</code>                    |
| Bezpośredni (Mem)      | <code>Imm</code>            | <code>Mem[Imm]</code>               |
| Pośredni (Mem)         | <code>(%Rb)</code>          | <code>Mem[%Rb]</code>               |
| Z przesunięciem        | <code>D(%Rb)</code>         | <code>Mem[%Rb + D]</code>           |
| Skalowany (Ind/scaled) | <code>D(%Rb, %Ri, S)</code> | <code>Mem[%Rb + %Ri * S + D]</code> |
| Skalowany (baseless)   | <code>(, %Ri, S)</code>     | <code>Mem[%Ri * S]</code>           |

**Legenda:** `Imm` / `D` to stała liczbowa, `%Ra` / `%Rb` to rejestr bazowy, `%Ri` to rejestr indeksowy, `a` / `S` to skala (tylko: 1, 2, 4 lub 8).

## 9. Flagi stanu i sterowanie

|                    |                                                               |                                                                    |
|--------------------|---------------------------------------------------------------|--------------------------------------------------------------------|
| <b>ZF</b>          | Zero. Wynik to 0 (np. argumenty są równe).                    |                                                                    |
| <b>SF</b>          | Sign. Wynik jest ujemny (MSB = 1).                            |                                                                    |
| <b>CF</b>          | Carry. Przepełnienie dla liczb <b>bez znaku</b> (unsigned).   |                                                                    |
| <b>OF</b>          | Overflow. Przepełnienie dla liczb <b>ze znakiem</b> (signed). |                                                                    |
|                    |                                                               |                                                                    |
| <b>cmp S1, S2</b>  | S2 − S1                                                       | Ustawia flagi jak odejmowanie, nie zapisuje wyniku.                |
|                    |                                                               |                                                                    |
| <b>test S1, S2</b> | S2 & S1                                                       | Ustawia flagi jak <b>AND</b> (np. test czy rejestr jest zerem).    |
|                    |                                                               |                                                                    |
| <b>jX dest</b>     | if(X) %rip ← dest                                             | Skok warunkowy (X = warunek).                                      |
|                    |                                                               |                                                                    |
| <b>setX D</b>      | if(X) D ← 1                                                   | Ustawia bajt (np. <code>%al</code> ) na 0 lub 1 na podstawie flag. |
|                    |                                                               |                                                                    |
| <b>cmovX S, D</b>  | if(X) D ← S                                                   | Warunkowe kopiowanie (optymalizacja zamiast skoku).                |

Sufiksy warunkowe (dla `jX`, `setX`, `cmovX`)

| Sufiks                      | Synonim   | Znaczenie                               |
|-----------------------------|-----------|-----------------------------------------|
| <b>e / z</b>                |           | Equal / Zero (równe / wynik to 0)       |
| <b>ne / nz</b>              |           | Not Equal / Not Zero (nierówne)         |
| <b>s</b>                    |           | Sign (wynik ujemny, <code>SF=1</code> ) |
| Liczby ze znakiem (signed)  |           |                                         |
| <b>g / ge</b>               |           | Greater / Greater or Equal              |
| <b>l / le</b>               |           | Less / Less or Equal                    |
| Liczby bez znaku (unsigned) |           |                                         |
| <b>a / ae</b>               | <b>nc</b> | Above / Above or Equal (No Carry)       |
| <b>b / be</b>               | <b>c</b>  | Below / Below or Equal (Carry)          |

Translacja struktur kontrolnych (C → ASM)

- if-else:** Realizowane przez skoki (`jX`) lub `cmovX`. `cmovX` unika kar za błędną predykcję skoku, ale oblicza zawsze obie ścieżki. Nie używa się go, gdy operacje są kosztowne obliczeniowo lub mają efekty uboczne (np. `x++`).
- Pętle do-while:** Ciało pętli jest na początku, na końcu znajduje się test i warunkowy skok w górę (`if(war) goto loop`).
- Pętle while:**
  - Jump-to-middle** (`-Og`): początkowy skok omija ciało pętli prosto do testu na końcu.
  - Do-while conversion** (`-O1`): początkowy strażnik (`if(!war) goto done`), po którym następuje zwykła pętla typu do-while.
- Pętle for:** Kompilator konwertuje je na pętle `while` według schematu: `init; while(war){ body; update it }`.
- Switch:** Używa **tablic skoków** dla osiągnięcia czasu skoku  $O(1)$  przy `case`. Realizowane przez pośredni skok z adresowaniem skalowanym: `jmp *.L4(,%rdi,8)`. Pomińcie `break` w języku C odpowiada fizycznemu brakowi instrukcji skoku kończącej dany blok w ASM (fall-through do kodu poniżej).

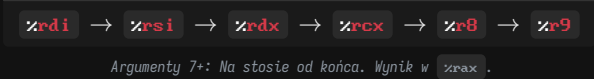
## 10. Procedury i stos

Stos w x86-64 rośnie **w dół** (w stronę niższych adresów). Rejestr `%rsp` wskazuje na **wierzchołek** stosu (najniższy zajęty adres). Wartości odkłada się używając `push` (zmniejsza `%rsp` o 8), a zdejmując `pop` (zwiększa `%rsp` o 8).

Przekazywanie sterowania

- `call dest`: Odkłada adres powrotu (kolejnej instrukcji) na stos i robi skok do `dest`.
- `ret`: Zdejmuje adres powrotu ze stosu i robi pod niego skok.

## 11. Konwencja Wywoływań (System V ABI)



Rejestry caller-saved vs callee-saved

| Caller-saved                                                                                                                                | Callee-saved                                                                                                         |
|---------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| (Wołający musi zapisać)<br>Mogą zostać nadpisane w funkcji. By je zachować, caller kładzie je na stos przed <code>call</code> .             | (Wołany musi przywrócić)<br>Muszą zachować stan. Callee musi zapisać je na stos i odtworzyć przed <code>ret</code> . |
| <code>%rax</code> , <code>%rdi</code> , <code>%rsi</code> ,<br><code>%rdx</code> , <code>%rcx</code> , <code>%r8</code> - <code>%r11</code> | <code>%rbx</code> , <code>%rbp</code> , <code>%r12</code> - <code>%r15</code>                                        |

Ramka stosu

Każde wywołanie funkcji otrzymuje własną ramkę na stosie (dzięki temu **rekurencja** działa naturalnie). **Alokacja ramki jest potrzebna, gdy:** brakuje rejestrów na zmienne (spilling), użyto lokalnej tablicy/struktury, lub pobrano adres zmiennej lokalnej (operator `&`).

| Prolog                                                                                                                                       | Epilog                                                                                                              |
|----------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| <pre>pushq %rbp ;<br/>zapisz stary base ptr<br/>movq %rsp, %rbp ;<br/>nowy base ptr = rsp<br/>subq \$32, %rsp ;<br/>alokacja 32 bajtów</pre> | <pre>addq \$32, %rsp ;<br/>zwolnij alokację<br/>popq %rbp ;<br/>przywróć base ptr<br/>ret ;<br/>skok powrotny</pre> |

**Sprzątanie przez `leave`:** Zastępuje `movq %rbp, %rsp` i `popq %rbp` w jednej instrukcji.

## 12. Architektura CPU

Przetwarzanie instrukcji dzieli się na 5 faz: **Fetch** (pobranie), **Decode** (dekodowanie/odczyt z rejestru), **Execute** (ALU), **Memory** (pamięć), **Write-back** (zapis do rejestru).

### Pipelining i hazardy

Nakładanie faz na siebie znacznie zwiększa throughput. Prowadzi to jednak do konfliktów (hazardów):

|            |                                                                                                                                                                                                                             |
|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Danych     | Odczyt po zapisie. Wynik instrukcji nie jest w rejestrze, a kolejna już go potrzebuje.<br><b>Rozwiązanie:</b> Forwarding/bypassing (przekazanie z ALU prosto na wejście następnej operacji) lub wstrzymanie (stall/bubble). |
| Sterowania | Instrukcja skoku wymusza odgadnięcie następnego adresu.<br><b>Rozwiązanie:</b> Branch prediction. Błąd kosztuje wyczyszczenie potoku (branch misprediction penalty).                                                        |

### Out-of-Order

- Nowoczesne procesory nie wykonują instrukcji sekwencyjnie.
- **Superskalarność:** Zdolność do wykonania wielu instrukcji w jednym cyklu zegara (posiadanie wielu jednostek wykonawczych).
  - **Register renaming:** Dynamiczne mapowanie rejestrów logicznych (`%rax`) na wiele rejestrów fizycznych. Eliminuje fałszywe zależności (nadpisywanie tego samego rejestru).
  - **Reorder buffer:** Gwarantuje, że instrukcje, choć liczone asynchronicznie, są ostatecznie zatwierdzane w oryginalnej kolejności programu, by zachować spójność.

## 13. Katalog instrukcji (AT&T)

| Opcode                      | Efekt                         | Opis                                                          |
|-----------------------------|-------------------------------|---------------------------------------------------------------|
| <code>mov S, D</code>       | $D \leftarrow S$              | Kopiuje wartość z S do D.                                     |
| <code>lea S, D</code>       | $D \leftarrow \text{addr}(S)$ | Oblicza adres S (bez czytania pamięci).                       |
| <code>add S, D</code>       | $D \leftarrow D + S$          | Dodawanie (ustawia flagi).                                    |
| <code>sub S, D</code>       | $D \leftarrow D - S$          | Odejmowanie (ustawia flagi).                                  |
| <code>imul S, D</code>      | $D \leftarrow D * S$          | Mnożenie liczb ze znakiem.                                    |
| <code>sar / shl k, D</code> | $D \ll = k$                   | Przesunięcie w lewo (mnożenie przez $2^k$ ).                  |
| <code>sar k, D</code>       | $D \gg = k$                   | Arytmetyczne w prawo (dzielenie). <b>Kopiuje znak.</b>        |
| <code>shr k, D</code>       | $D \gg = k$                   | Logiczne w prawo. Dopełnia zerami.                            |
| <code>and / or S, D</code>  | $D \leftarrow D \& /   S$     | Operacje bitowe (ustawiają flagi).                            |
| <code>xor S, D</code>       | $D \leftarrow D \wedge S$     | Często używane jako <code>xor %rax, %rax</code> do zerowania. |

### Porównania i sterowanie

|                          |                                             |                                                       |
|--------------------------|---------------------------------------------|-------------------------------------------------------|
| <code>cmp S1, S2</code>  | $S2 - S1$                                   | Ustawia flagi jak odejmowanie, nie zapisuje wyniku.   |
| <code>test S1, S2</code> | $S2 \& S1$                                  | Ustawia flagi jak <b>AND</b> .                        |
| <code>jX dest</code>     | $\text{if}(X) \%rip \leftarrow \text{dest}$ | Skok warunkowy (X = warunek).                         |
| <code>setX D</code>      | $\text{if}(X) D \leftarrow 1$               | Ustawia najmłodszy bajt na 0 lub 1 na podstawie flag. |
| <code>cmovX S, D</code>  | $\text{if}(X) D \leftarrow S$               | Warunkowe kopiowanie (optymalizacja zamiast skoku).   |

### Stos i zarządzanie procedurami

|                        |                                                        |                                                       |
|------------------------|--------------------------------------------------------|-------------------------------------------------------|
| <code>push S</code>    | $\%rsp - = 8$<br>$M[\%rsp] \leftarrow S$               | Odkłada na stos (zmniejsza <code>%rsp</code> ).       |
| <code>pop D</code>     | $D \leftarrow M[\%rsp]$<br>$\%rsp + = 8$               | Zdejmuje ze stosu do D (zwiększa <code>%rsp</code> ). |
| <code>call dest</code> | $\text{push } \%rip$<br>$\%rip \leftarrow \text{dest}$ | Skok do funkcji (zapisuje adres powrotu na stosie).   |
| <code>ret</code>       | $\text{pop } \%rip$                                    | Zdejmuje adres powrotu ze stosu i skacze pod niego.   |
| <code>leave</code>     | $\%rsp \leftarrow \%rbp$<br>$\text{pop } \%rbp$        | Sprząta ramkę stosu (odwrotność prologu).             |

Operacje wektorowe

Rozszerzenie AVX. Przedrostek **v** (nie niszczy źródeł).

|                                    |                            |                                                                                                         |
|------------------------------------|----------------------------|---------------------------------------------------------------------------------------------------------|
| <code>vmovaps / ups S, D</code>    | $D \leftarrow S$           | Kopiuje wektor. <b>a</b> (aligned - szybkie, pamięć dzieli się przez wyrównanie), <b>u</b> (unaligned). |
| <code>vaddps / pd S1, S2, D</code> | $D \leftarrow S2 + S1$     | Dodawanie wielokrotne (packed). <b>ps</b> (single-float), <b>pd</b> (double).                           |
| <code>vmulps / pd S1, S2, D</code> | $D \leftarrow S2 * S1$     | Mnożenie wektorowe.                                                                                     |
| <code>vfmadd231ps S1, S2, D</code> | $D \leftarrow S2 * S1 + D$ | <b>Fused Multiply-Add.</b> Mnoży i dodaje do <b>D</b> w jednym kroku.                                   |

**Uwaga o sufiksach:** Instrukcje przyjmują sufiks określający rozmiar danych: **b** (8-bit), **w** (16-bit), **l** (32-bit), **q** (64-bit).  
Np. `movl` kopiuje 32 bity (i automatycznie zeruje górną połowę 64-bitowego rejestru!).